

AI Assisted Coding

2303A510A0(Vishnu vardhan)

[Batch-14]

Assignment -12.4

Lab- Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms.

Task-1: Bubble Sort for Ranking Exam

Prompt: Write Bubble Sort in Python with comments and time complexity. Include sample input/output.

Code:

```
1 > ass 11.1 > Ai-Ass-12.4.py > ...
def bubble_sort(scores):
    n = len(scores)

    for i in range(n):
        swapped = False # Track if swap happens

        for j in range(0, n - i - 1):
            if scores[j] > scores[j + 1]:
                # Swap elements
                scores[j], scores[j + 1] = scores[j + 1], scores[j]
                swapped = True

        # Stop early if already sorted
        if not swapped:
            break

    return scores

# User Input
scores = list(map(int, input("Enter student scores separated by space: ").split()))
sorted_scores = bubble_sort(scores)

print("Sorted Scores:", sorted_scores)
```

Output:-

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\vishn\OneDrive\Desktop\AI> & C:/Users/vishn/AppData/Local/
e/Desktop/AI/as 8.1/ass 11.1/Ai-Ass-12.4.py"
● Enter student scores separated by space: 99 93 94 95 96 98
Sorted Scores: [93, 94, 95, 96, 98, 99]
○ PS C:\Users\vishn\OneDrive\Desktop\AI> █
```

Explanation

- Compares adjacent elements.
- Swaps if they are in wrong order.
- Stops early if no swaps occur.
- Suitable for small datasets.

Time Complexity

- Best: $O(n)$ (already sorted)
- Average: $O(n^2)$
- Worst: $O(n^2)$

Task-2: Improving Sorting for Nearly Sorted

Attendance Records

Prompt: Suggest a better algorithm for nearly sorted data and implement Insertion Sort. Compare with Bubble Sort.

Code:

```
26 def insertion_sort(arr):
27     for i in range(1, len(arr)):
28         key = arr[i] # Current element
29         j = i - 1
30
31         # Shift larger elements to right
32         while j >= 0 and arr[j] > key:
33             arr[j + 1] = arr[j]
34             j -= 1
35
36         arr[j + 1] = key # Insert at correct position
37
38     return arr
39
40
41 # User Input
42 attendance = list(map(int, input("Enter roll numbers: ").split()))
43 sorted_attendance = insertion_sort(attendance)
44
45 print("Sorted Attendance:", sorted_attendance)
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL TORIS
PS C:\Users\vishn\OneDrive\Desktop\AI> & C:/Users/vishn/AppData/Local/Programs/Python/Python38-64/Python.exe/Desktop/AI/as 8.1/ass 11.1/Ai-Ass-12.4.py
Enter roll numbers: 1 2 43 64 62 92 18 34
Sorted Attendance: [1, 2, 18, 34, 43, 62, 64, 92]
PS C:\Users\vishn\OneDrive\Desktop\AI>
```

Explanation

- Picks one element at a time.
- Inserts into correct sorted position.
- Very efficient for nearly sorted data.

Time Complexity

- Best: $O(n)$
- Average: $O(n^2)$
- Worst: $O(n^2)$

Task-3: Searching Student Records in a Database

Prompt: Implement Linear and Binary Search with docstrings. Compare their time complexities.

Code: Linear Search

```
s 8.1 > ass 11.1 > Ai-Ass-12.4.py > ...
46
47 v def linear_search(arr, target):
48 v     """
49     Performs Linear Search on unsorted list.
50     Returns index if found, otherwise -1.
51     """
52 v     for i in range(len(arr)):
53 v         if arr[i] == target:
54             return i
55     return -1
56
57
58 arr = list(map(int, input("Enter roll numbers: ").split()))
59 target = int(input("Enter roll number to search: "))
60
61 result = linear_search(arr, target)
62
63 v if result != -1:
64     print("Found at index:", result)
65 v else:
66     print("Not Found")
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\vishn\OneDrive\Desktop\AI> & C:/Users/vishn/AppData/Local/Programs/Python/
e/Desktop/AI/as 8.1/ass 11.1/Ai-Ass-12.4.py"
Enter roll numbers: 1 4 2 6 3 8 9
Enter roll number to search: 3
Found at index: 4
PS C:\Users\vishn\OneDrive\Desktop\AI> 
```

Linear Search – Explanation

Linear Search checks each element one by one.

It starts from index 0 and compares every roll number with the target.

If a match is found, it returns the index.

If the loop finishes without finding it, the element is not present.

Time Complexity

- Best Case: $O(1)$ (element found at first position)
- Average Case: $O(n)$
- Worst Case: $O(n)$ (element at last position or not found)

Code: Binary Search (Sorted Data Only)

```

1 / ass 11.1 / AI Ass 12.4.py / binary_search
2
3 def binary_search(arr, target):
4
5     low = 0
6     high = len(arr) - 1
7
8
9     while low <= high:
10         mid = (low + high) // 2
11
12         if arr[mid] == target:
13             return mid
14         elif arr[mid] < target:
15             low = mid + 1
16         else:
17             high = mid - 1
18
19     return -1
20
21 arr = sorted(list(map(int, input("Enter roll numbers: ").split()))))
22 target = int(input("Enter roll number to search: "))
23
24 result = binary_search(arr, target)
25
26 print("Sorted List:", arr)
27
28 if result != -1:
29     print("Found at index:", result)
30 else:
31     print("Not Found")

```

Output:-

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\vishn\OneDrive\Desktop\AI> & C:/Users/vishn/AppData/Local/Programs/Python/P
e/Desktop/AI/as 8.1/ass 11.1/Ai-Ass-12.4.py"
Enter roll numbers: 1 5 2 7 9 3 8
Enter roll number to search: 7
Sorted List: [1, 2, 3, 5, 7, 8, 9]
Found at index: 4
PS C:\Users\vishn\OneDrive\Desktop\AI>

```

Binary Search – Explanation

Binary Search works only on sorted data.

It finds the middle element.

If target equals middle → found.

If target is smaller → search left half.

If target is larger → search right half.

This process continues until element is found or search space becomes empty.

Time Complexity

- Best Case: $O(1)$
- Average Case: $O(\log n)$
- Worst Case: $O(\log n)$
-

Task-4: Choosing Between Quick Sort and Merge Sort for Data Processing

Prompt: Complete Quick Sort and Merge Sort recursively. Compare their complexities and use cases.

Code: Quick Sort

```
8.1 > ass 11.1 > Ai-Ass-12.4.py > ...
-
02 def quick_sort(arr):
03     """
04     Quick Sort using recursion.
05     Divides array based on pivot element.
06     """
07     if len(arr) <= 1:
08         return arr
09
10     pivot = arr[len(arr) // 2]
11
12     left = [x for x in arr if x < pivot]
13     middle = [x for x in arr if x == pivot]
14     right = [x for x in arr if x > pivot]
15
16     return quick_sort(left) + middle + quick_sort(right)
17
18
19 arr = list(map(int, input("Enter numbers: ").split()))
20 print("Quick Sort Result:", quick_sort(arr))
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\vishn\OneDrive\Desktop\AI> & C:/Users/vishn/AppData/Local/Programs/Python/Python38-6/Desktop/AI/as 8.1/ass 11.1/Ai-Ass-12.4.py
Enter numbers: 2 5 6 9 3 7 4
Quick Sort Result: [2, 3, 4, 5, 6, 7, 9]
PS C:\Users\vishn\OneDrive\Desktop\AI> 
```

Explanation:

Select a pivot element.

Divide the array into:

- Elements smaller than pivot
- Elements equal to pivot
- Elements greater than pivot

Recursively apply Quick Sort to left and right parts.

Time Complexity

- Best Case: $O(n \log n)$ (balanced partitions)
- Average Case: $O(n \log n)$
- Worst Case: $O(n^2)$ (already sorted + bad pivot)

Code: Merge Sort

```

ass 11.1 / AI-Ass-12.4.py / merge_sort

def merge_sort(arr):
    """
    Merge Sort divides array into halves,
    recursively sorts and merges them.
    """
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

arr = list(map(int, input("Enter numbers: ").split()))
print("Merge Sort Result:", merge_sort(arr))

```

Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\vishn\OneDrive\Desktop\AI> & C:/Users/vishn/AppData/Local/Programs/Python/Python38-64/Scripts/python.exe C:/Users/vishn/Desktop/AI/as 8.1/ass 11.1/Ai-Ass-12.4.py
Enter numbers: 3 6 2 8 1 9
Merge Sort Result: [1, 2, 3, 6, 8, 9]
PS C:\Users\vishn\OneDrive\Desktop\AI>

```

Explanation:

Merge Sort also uses Divide and Conquer:

1. Divide array into two halves.
2. Recursively sort each half.
3. Merge the sorted halves into one sorted array.

Time Complexity

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n \log n)$

Task-5: Optimizing a Duplicate Detection

Algorithm

Prompt: Analyze nested-loop duplicate detection and optimize using sets. Compare $O(n^2)$ vs $O(n)$.

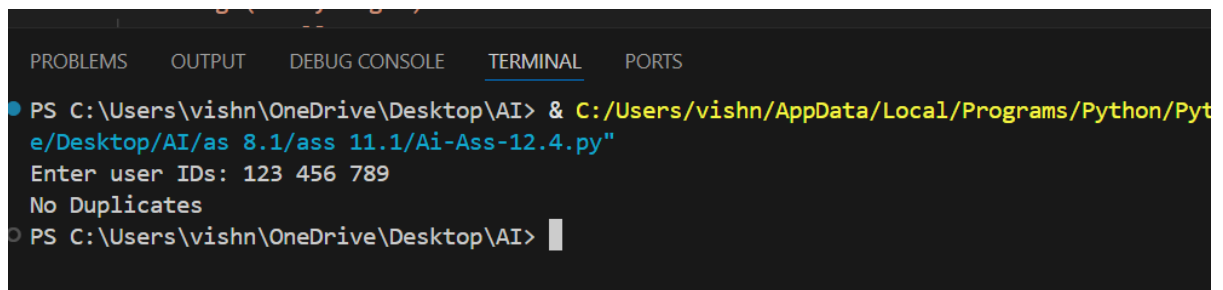
Code: Brute Force

```

132 print("Merge Sort Result:", merge_sort(arr))"""
133
134 def has_duplicates_bruteforce(arr):
135     for i in range(len(arr)):
136         for j in range(i + 1, len(arr)):
137             if arr[i] == arr[j]:
138                 return True
139     return False
140
141
142 arr = list(map(int, input("Enter user IDs: ").split()))
143
144 if has_duplicates_bruteforce(arr):
145     print("Duplicates Found")
146 else:
147     print("No Duplicates")

```

Output:



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\vishn\OneDrive\Desktop\AI> & C:/Users/vishn/AppData/Local/Programs/Python/Python311/Scripts/python C:/Users/vishn/Desktop/AI/as 8.1/ass 11.1/Ai-Ass-12.4.py
Enter user IDs: 123 456 789
No Duplicates
PS C:\Users\vishn\OneDrive\Desktop\AI>

```

Time complexity:

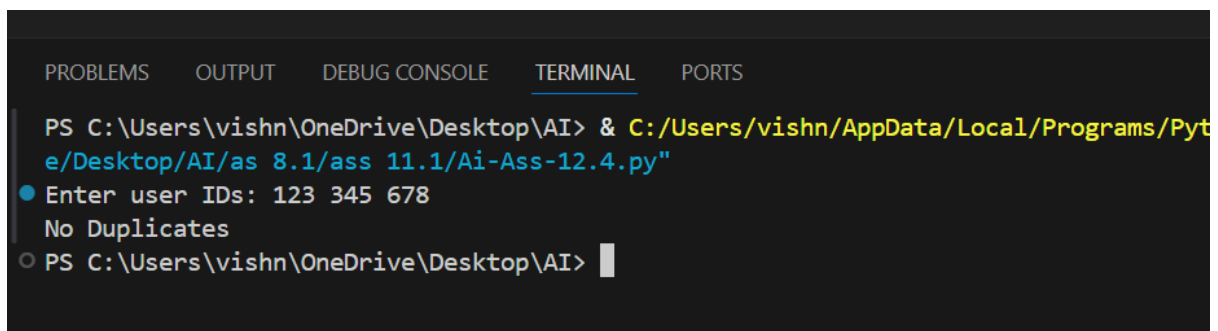
Best Case: $O(n^2)$

Worst Case: $O(n^2)$

Code: Optimized Using Set

```
147     print("No Duplicates")
148
149 def has_duplicates_optimized(arr):
150     seen = set()
151
152     for element in arr:
153         if element in seen:
154             return True
155         seen.add(element)
156
157     return False
158
159
160 arr = list(map(int, input("Enter user IDs: ").split()))
161
162 if has_duplicates_optimized(arr):
163     print("Duplicates Found")
164 else:
165     print("No Duplicates")
```

Output:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\vishn\OneDrive\Desktop\AI> & C:/Users/vishn/AppData/Local/Programs/Python/Python38-64/Scripts/python.exe C:/Users/vishn/Desktop/AI/as 8.1/ass 11.1/Ai-Ass-12.4.py
● Enter user IDs: 123 345 678
No Duplicates
○ PS C:\Users\vishn\OneDrive\Desktop\AI>
```

Explanation:

Nested loops $\rightarrow O(n^2)$

Set lookup $\rightarrow O(1)$

Total optimized $\rightarrow O(n)$

Time complexity:

Best Case: $O(n)$

Worst Case: $O(n)$